

Raport o stanie gałęzi

quantum_statesDB

Projekt IDUB_2026

Audyt kodu i plan działań

https://github.com/e-mcQuantum-AI/IDUB_2026/tree/quantum_statesDB

28 marca 2026

Spis treści

1	Opis gałęzi	2
2	Ocena nowego kodu	2
2.1	Co działa dobrze	2
2.2	Poprawność fizyczna	3
3	Wykryte błędy i problemy	3
3.1	Błędy krytyczne	3
3.2	Problemy istotne	3
3.3	Problemy mniejsze	4
4	Nowy kod źródłowy	4
5	Plan działań	6
5.1	Faza DB-1 — Naprawy krytyczne	6
5.1.1	DB-1.2 Rozszerzenie samplera o wszystkie stany	6
5.1.2	DB-1.3 Jawna konfiguracja rozdzielczości	7
5.2	Faza DB-2 — Ulepszenia	8
5.3	Faza DB-3 — Jakość i testy	8
6	Relacja z gałęzią main	10
7	Podsumowanie	10

1 Opis gałęzi

Gałąź `quantum_statesDB` rozszerza gałąź `main` o moduł generowania zbiorów danych numerycznych (tablic NumPy) zamiast obrazów PNG. Wprowadza nową architekturę generatora opartą na trzech komponentach:

1. `src/dataset/state_sampler.py` — funkcja losująca stany kwantowe z etykietami,
2. `src/dataset/generator.py` — klasa `DatasetGenerator` łącząca sampler z pipeline'em pomiarowym,
3. `scripts/generate_dataset.py` — skrypt uruchamiający generowanie 1000 próbek i zapisujący wynik do `.npz`.

Jest to istotny krok w kierunku modułu ML — dane w formacie tablicowym (NumPy) są bezpośrednio gotowe do użycia jako dane treningowe, w przeciwieństwie do obrazów PNG z gałęzi `main`.

Nowe pliki: 3 pliki, ~60 linii kodu.

Zmiana w istniejącym kodzie: 1 linia w `generate.py` (dodanie folderu `noisy`).

2 Ocena nowego kodu

2.1 Co działa dobrze

Mocne strony gałęzi

- **Dobra architektura:** rozdzielenie odpowiedzialności na sampler, generator i skrypt — czytelny wzorzec *strategy* (strategia losowania jest wymienną funkcją).
- **Użycie istniejących klas:** `generate_dataset.py` korzysta z `WignerMeasurement`, `LossChannel`, `MeasurementPipeline` — spójność z resztą projektu.
- **Format NumPy:** zapis do `.npz` jest bezpośrednio gotowy do uczenia maszynowego (PyTorch `Dataset`, scikit-learn itp.).
- **Funkcja lambda jako sampler:** elastyczny wzorzec — łatwo podmienić `simple_sampler` na bardziej złożony.

Tabela 1: Poprawność implementacji na gałęzi `quantum_statesDB`.

Komponent	Ocena	Status
<code>DatasetGenerator.generate()</code>	Poprawna logika	✓
<code>simple_sampler()</code>	Fizycznie poprawne stany	✓
Pipeline z szumem	Poprawne użycie <code>LossChannel</code>	✓

2.2 Poprawność fizyczna

3 Wykryte błędy i problemy

3.1 Błędy krytyczne

DB-B1. Brak plików `__init__.py` w `src/dataset/`

Nowy katalog `src/dataset/` nie ma pliku `__init__.py`. `Import from src.dataset.generator import DatasetGenerator` może nie działać. To ten sam problem co B1 z gałęzi `main`, rozszerzony o nowy katalog.

DB-B2. Sampler obsługuje tylko 2 z 7 stanów

`simple_sampler()` losuje tylko próżnię i stan koherentny. Brakuje 5 stanów zaimplementowanych w projekcie: Fock, kot, termiczny, dwumianowy, GKP. Przy 1000 próbek zbiór danych będzie zawierał tylko 2 klasy — niewystarczające do treningu klasyfikatora rozróżniającego wszystkie typy stanów.

DB-B3. Niespójność `cutoff` między generatorem a pipeline’em

Skrypt używa `cutoff=40`, ale domyślna rozdzielczość `WignerMeasurement` to `resolution=64`. Sam `cutoff` nie jest przekazywany do `WignerMeasurement` — to osobne parametry, ale brak jawnej konfiguracji rozdzielczości wyjściowego obrazu Wignera oznacza, że rozmiar danych zależy od ukrytej wartości domyślnej.

3.2 Problemy istotne

DB-P1. Brak `__init__.py` w `scripts/` — katalog `scripts/` nie jest pakietem Pythona, co jest akceptowalne (to skrypt uruchamiany bezpośrednio), ale wymaga uruchomienia z poziomu katalogu głównego: `python -m scripts.generate_dataset` lub ustawienia `PYTHONPATH`.

DB-P2. Brak walidacji w `DatasetGenerator` — metoda `generate()` nie sprawdza, czy `n_samples > 0`, nie obsługuje wyjątków z pipeline’u, nie pokazuje postępu generowania.

DB-P3. Etykiety numeryczne bez mapowania — `simple_sampler` zwraca 0 (próżnia) i 1 (koherentny), ale nigdzie nie jest zapisane mapowanie numer→nazwa. Przy rozszerzeniu na więcej stanów łatwo o pomyłkę.

DB-P4. Brak adnotacji typów i docstringów — żaden z 3 nowych plików nie ma type hintów ani docstringów.

DB-P5. Brak testów — nowy kod nie ma testów jednostkowych.

DB-P6. Stary `generate.py` nie jest używany — plik `src/physics/generate.py` (z błędem B2 z `main`) nadal istnieje i jest wywoływany bezpośrednio (`generate_dataset(n_samples=5)` na końcu pliku). Nowy generator go nie zastępuje — powstaje zamieszanie, który skrypt jest aktualny.

3.3 Problemy mniejsze

- **DB-P7.** Plik wyjściowy `dataset_v0.npz` zapisywany w katalogu roboczym — brak konfigurowalnej ścieżki.
- **DB-P8.** Brak losowego ziarna (`random.seed` / `np.random.seed`) — wyniki nie są odtwarzalne.
- **DB-P9.** `generate.py` zmieniony (dodano folder `noisy`), ale folder ten nigdzie nie jest używany.

4 Nowy kod źródłowy

```
1 import random
2 from src.physics.state.vacuum import VacuumState
3 from src.physics.state.coherent import CoherentState
4
5 def simple_sampler(cutoff):
6
7     choice = random.choice(["vacuum", "coherent"])
8
9     if choice == "vacuum":
10         return VacuumState(cutoff), 0
11
12     else:
13         alpha = random.uniform(0.5, 2.0)
14         return CoherentState(alpha, cutoff), 1
```

Listing 1: `src/dataset/state_sampler.py` — sampler losujący stany.

```
1 import numpy as np
2
3 class DatasetGenerator:
4
5     def __init__(self, state_sampler, pipeline):
6         self.state_sampler = state_sampler
7         self.pipeline = pipeline
8
9     def generate(self, n_samples):
10         X = []
11         y = []
12
13         for _ in range(n_samples):
14             state, label = self.state_sampler()
15             W = self.pipeline.run(state)
16
17             X.append(W)
18             y.append(label)
19
20         return np.array(X), np.array(y)
```

Listing 2: src/dataset/generator.py — generator zbiorów danych.

```
1 from src.physics.measurement.wigner import WignerMeasurement
2 from src.physics.noise.loss import LossChannel
3 from src.physics.measurement.pipeline import MeasurementPipeline
4 from src.dataset.generator import DatasetGenerator
5 from src.dataset.state_sampler import simple_sampler
6 import numpy as np
7
8 if __name__ == '__main__':
9     cutoff = 40
10
11     measurement = WignerMeasurement()
12     noise = LossChannel(cutoff=cutoff, gamma=0.1)
13
14     pipeline = MeasurementPipeline(measurement, noise)
15
16     generator = DatasetGenerator(
17         lambda: simple_sampler(cutoff),
18         pipeline
```

```

19 )
20
21 X, y = generator.generate(1000)
22
23 np.savez("dataset_v0.npz", X=X, y=y)

```

Listing 3: scripts/generate_dataset.py — skrypt generujący zbiór danych.

5 Plan działań

5.1 Faza DB-1 — Naprawy krytyczne

Tabela 2: Faza DB-1: naprawy krytyczne.

#	Zadanie	Czas
DB-1.1	Dodać <code>__init__.py</code> w <code>src/dataset/</code>	10 min
DB-1.2	Rozszerzyć <code>simple_sampler</code> o wszystkie 7 stanów	2 godz.
DB-1.3	Jawnie przekazywać <code>resolution</code> do <code>WignerMeasurement</code> w skrypcie	15 min

5.1.1 DB-1.2 Rozszerzenie samplera o wszystkie stany

Obecna implementacja losuje tylko 2 stany (próżnia, koherentny). Aby zbiór danych był użyteczny do klasyfikacji, sampler powinien obejmować wszystkie 7 typów:

```

1 import random
2 from src.physics.state.vacuum import VacuumState
3 from src.physics.state.fock import FockState
4 from src.physics.state.coherent import CoherentState
5 from src.physics.state.cat import CatState
6 from src.physics.state.thermal import ThermalState
7 from src.physics.state.binomial import BinomialState
8 from src.physics.state.gkp import GKPState
9
10 # Mapowanie etykiet: numer -> nazwa stanu
11 LABEL_MAP = {
12     0: "vacuum",
13     1: "fock",
14     2: "coherent",
15     3: "cat",
16     4: "thermal",

```

```

17     5: "binomial",
18     6: "gkp",
19 }
20
21 def full_sampler(cutoff: int):
22     """Losuje stan kwantowy ze wszystkich 7 typow.
23
24     Returns:
25         (state, label): para (obiekt QuantumState, int).
26     """
27
28     label = random.randint(0, 6)
29     name = LABEL_MAP[label]
30
31     if name == "vacuum":
32         return VacuumState(cutoff), label
33     elif name == "fock":
34         n = random.randint(1, min(7, cutoff - 1))
35         return FockState(n, cutoff), label
36     elif name == "coherent":
37         alpha = random.uniform(0.5, 3.0)
38         return CoherentState(alpha, cutoff), label
39     elif name == "cat":
40         alpha = random.uniform(1.5, 3.0)
41         return CatState(alpha, cutoff), label
42     elif name == "thermal":
43         n_th = random.uniform(0.5, 4.0)
44         return ThermalState(n_th, cutoff), label
45     elif name == "binomial":
46         N = random.randint(2, min(6, cutoff - 1))
47         p = random.uniform(0.2, 0.8)
48         return BinomialState(N, p, cutoff), label
49     elif name == "gkp":
50         delta = random.uniform(0.25, 0.5)
51         return GKPSState(cutoff, delta), label

```

Listing 4: Proponowany rozszerzony sampler.

Kryterium: Wygenerowany zbiór zawiera wszystkie 7 klas. Rozkład klas jest w przybliżeniu równomierny.

5.1.2 DB-1.3 Jawna konfiguracja rozdzielczości

```

1 # PRZED:
2 measurement = WignerMeasurement() # resolution=64 (ukryte)
3
4 # PO:
5 RESOLUTION = 64
6 measurement = WignerMeasurement(x_max=5, resolution=RESOLUTION)

```

Listing 5: Jawne przekazanie rozdzielczości.

5.2 Faza DB-2 — Ulepszenia

Tabela 3: Faza DB-2: ulepszenia.

#	Zadanie	Czas
DB-2.1	Dodać mapowanie etykiet (LABEL_MAP) i zapis do .npz	30 min
DB-2.2	Dodać postęp generowania (tqdm lub print)	15 min
DB-2.3	Dodać odtwarzalność (random.seed, np.random.seed)	15 min
DB-2.4	Sparametryzować ścieżkę wyjściową i liczbę próbek (argparse)	1 godz.
DB-2.5	Usunąć lub oznaczyć stary src/physics/generate.py jako przestarzały	15 min
DB-2.6	Usunąć nieużywany folder noisy z setup_folders()	5 min

5.3 Faza DB-3 — Jakość i testy

Tabela 4: Faza DB-3: jakość kodu i testy.

#	Zadanie	Czas
DB-3.1	Dodać adnotacje typów do generator.py i state_sampler.py	30 min
DB-3.2	Dodać docstringi	30 min
DB-3.3	Testy: DatasetGenerator zwraca tablice o poprawnych kształtach	1 godz.
DB-3.4	Testy: sampler zwraca poprawne typy i etykiety z zakresu	1 godz.
DB-3.5	Testy: wygenerowany zbiór jest fizycznie poprawny (brak NaN/Inf)	30 min

Przykłady testów:


```
1 class TestDatasetGenerator:
2     def test_output_shape(self):
3         """Wygenerowane tablice maja poprawne kształty."""
4         wm = WignerMeasurement(x_max=5, resolution=32)
5         pipeline = MeasurementPipeline(measurement=wm)
6         gen = DatasetGenerator(
7             lambda: (VacuumState(20), 0), pipeline
8         )
9         X, y = gen.generate(5)
10        assert X.shape == (5, 32, 32)
11        assert y.shape == (5,)
12
13    def test_no_nan(self):
14        """Brak wartosci NaN w danych."""
15        wm = WignerMeasurement(x_max=5, resolution=32)
16        pipeline = MeasurementPipeline(measurement=wm)
17        gen = DatasetGenerator(
18            lambda: (VacuumState(20), 0), pipeline
19        )
20        X, y = gen.generate(3)
21        assert np.isfinite(X).all()
22
23 class TestStateSampler:
24     def test_returns_valid_types(self):
25         """Sampler zwraca (QuantumState, int)."""
26         state, label = simple_sampler(20)
27         assert hasattr(state, 'density_matrix')
28         assert isinstance(label, int)
29
30     def test_label_range(self):
31         """Etykiety w zakresie [0, 1]."""
32         labels = [simple_sampler(20)[1] for _ in range(100)]
33         assert set(labels) == {0, 1}
```

Listing 6: Testy dla modułu `dataset`.

6 Relacja z gałęzią main

Zalecenia dotyczące scalenia

1. **Przed merge'em:** najpierw wykonać naprawy z Fazy 1 gałęzi main (pliki `__init__.py`, literówka `binomial.py`, dziedziczenie kanałów) — bez nich kod z `quantum_statesDB` nie uruchomi się poprawnie.
2. **Rozszerzyć sampler** (DB-1.2) przed scaleniem — zbiór z 2 klasami ma ograniczoną wartość.
3. **Usunąć stary generate.py** lub oznaczyć jako przestarzały (DB-2.5) — dwa generatory tworzą zamieszanie.
4. **Scalenie:** po naprawach z obu stron, merge `quantum_statesDB` → `main` powinien przebiec bez konfliktów (nowe pliki, minimalna zmiana w istniejących).

7 Podsumowanie

Gałąź `quantum_statesDB` wprowadza **dobry fundament architektoniczny** dla generowania zbiorów danych ML: rozdzielenie samplera, generatora i skryptu, format NumPy gotowy do treningu. Jest to krok we właściwym kierunku.

Główne braki: sampler obsługuje tylko 2 z 7 stanów, brak `__init__.py`, brak mapowania etykiet, brak testów.

Szacowany nakład: ok. **8 godzin** na wszystkie 3 fazy (DB-1 → DB-2 → DB-3).

Zalecana kolejność: naprawy main (Faza 1) → naprawy `quantum_statesDB` (DB-1) → scalenie → DB-2, DB-3.